

SOC Design Flow

SOC Design Flow

- The different and conflicting requirements in SOC design are of increasing design size, deep sub-micron effects (DSM) and the necessity for shorter and predictable implementation times.
- To meet these requirements, SOC designs are implemented by following a design flow: -
 - Bottom–Up design flow
 - Top-down design flow

Bottom-Up Design Approach

- The design team starts by partitioning the system design into various subsystem and system components (blocks). The subsystems are targeted to ASICs, FPGAs, or microprocessors.
- Since these subsystem designs are usually on the critical path to completing the design, the team starts on these immediately, developing the other system components in parallel. Each block is designed and verified based on its own requirements. When all blocks are complete, system verification begins.
- The bottom-up design approach has the **advantages of focusing on the initial product delivery and of allowing work to begin immediately on critical portions of the system.**
- With this approach, however, system-level design errors do not surface until late in the design cycle and may require costly design iterations— **Disadvantage**
- Furthermore, while related products can reuse lower-level components, they cannot have any system-level similarities in design architecture, intellectual property, or verification environment - **Disadvantage**
- Finally, bottom-up design requires commitment to a semiconductor technology process early on and hinders the ability to reuse designs in other technology processes- **Disadvantage**

Top-Down Design Approach

- The alternative approach is the top-down design approach.
- In this approach, the **design team invests** time up front **in developing system-level models and a verification environment**.
- Using the system models, the team is able to analyse trade-offs in system performance, feature set, partitioning, and packaging.
- Furthermore, a system-level verification environment ensures that system requirements are met and provides the infrastructure for verifying the subsystems and system components.
- The top-down design approach **results in higher confidence that the completed design will meet the original schedule and system specifications**
- Basing the starting point of the system design on a single verified model ensures that critical design issues surface early in the process and reduces false starts in the concurrent design of ASICs, PCBs, and systems.

Cont...

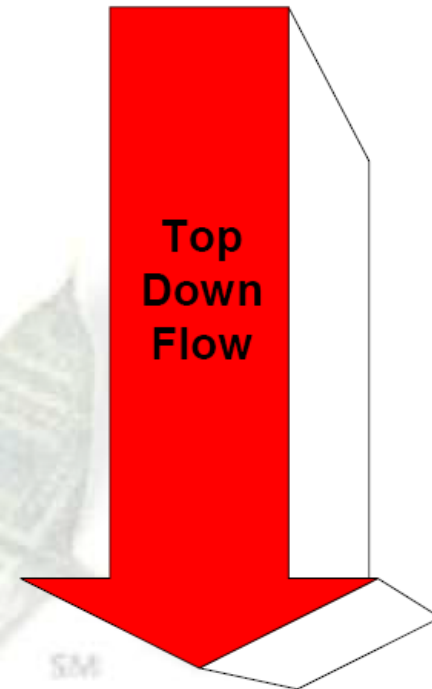
- The design team can discover and manage system-level issues up front, rather than having to redesign the system at the end of the design cycle--**Advantage**
- Because each subsystem is designed and verified within the context of the system verification environment, the overall system functionality is preserved--**Advantage.**
- The top-down design approach also effectively leverages the initial product development in the design of related products. The related projects begin with the system environment in place.
- The design team can reuse and re-verify alternative designs, packages, or implementations without having to rebuild a new context or infrastructure.

Basic Principles of the Top-Down Approach

- Understanding the basic principles of top-down design is the first step toward implementing the best design practices. The top-down design approach is based on the following principles:
 - Use a hardware description language (HDL) or other high-level programming language to create system and subsystem models as well as reusable cores.
 - Validate designs early by developing a system-level verification environment up front. A system verification environment includes a set of test-benches and models, and a detailed, formal test plan for validation of the system. The models and test benches are a “golden” representation of the design that the team can use to qualify the design of components.
 - Develop a design for test (DFT) strategy.
 - Provide for consistent data flow between logical and physical design processes for deep submicron ICs and high-speed PCBs

Top-Down Design

- **System specification**
- **Refine architecture, algorithms**
- **Decompose into blocks**
- **Design or select macros**
- **Integrate macros**
- **Deliver to next level integration**
- **Verify**



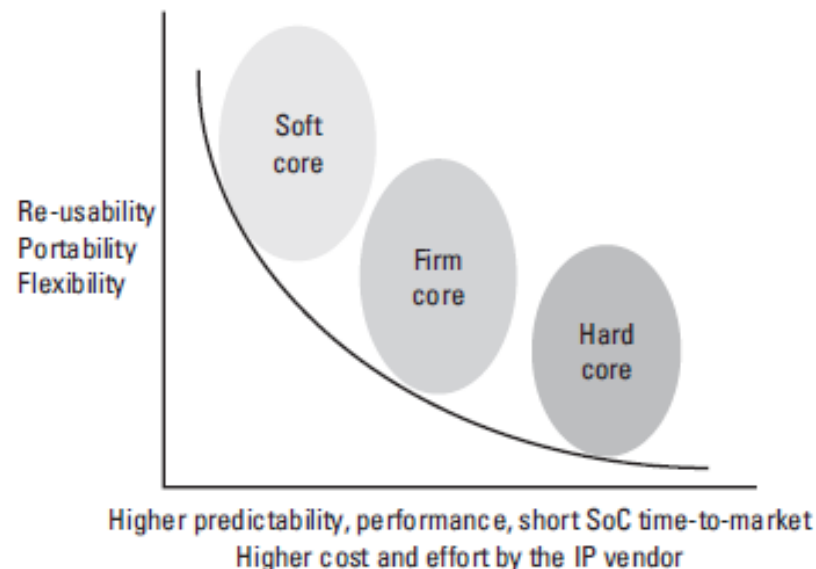
SOC Design Process

- Addressing these problems concurrently
 - Functionality,
 - Timing,
 - Physical design and
 - Verification
- Incrementally improving as design converges
- Top-down to combination of top-down and bottom-up
 - Bottom-up with critical low-level blocks, reuse soft or hard macros

Soft cores: These are reusable blocks in the form of synthesizable RTL descriptions or netlists of generic library elements. This implies that the user of soft core (macro) is responsible for the actual implementation and layout.

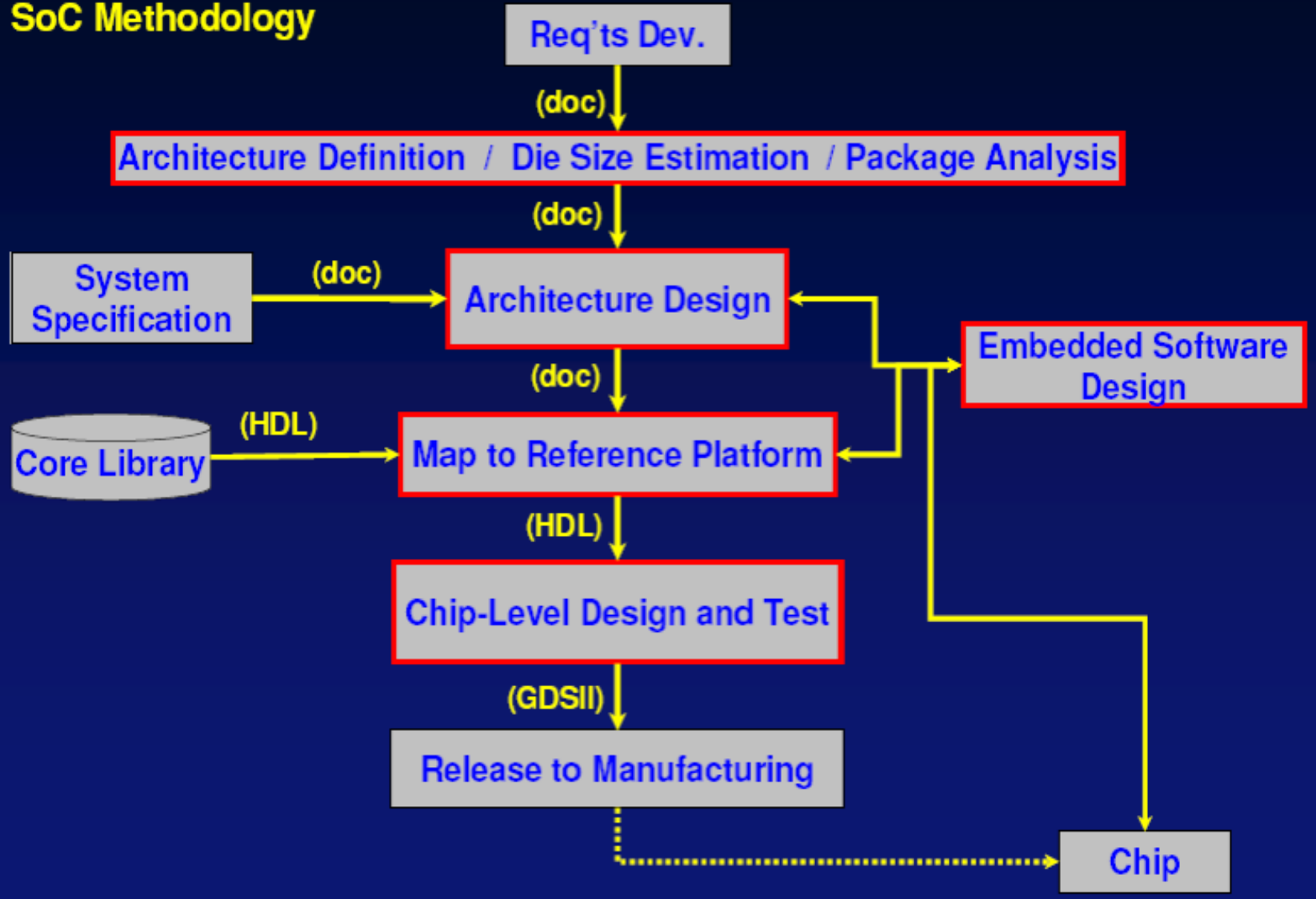
Firm cores: These are reusable blocks that have been structurally and topologically optimised for performance and area through floor planning and placement, perhaps using a range of process technologies. These exist as synthesised code or as a netlist of generic library elements.

Hard cores: These are reusable blocks optimised for performance, power, and size, and mapped to a specific process technology. These exist as a fully placed and routed netlist and as a fixed layout, such as in GDSII format.



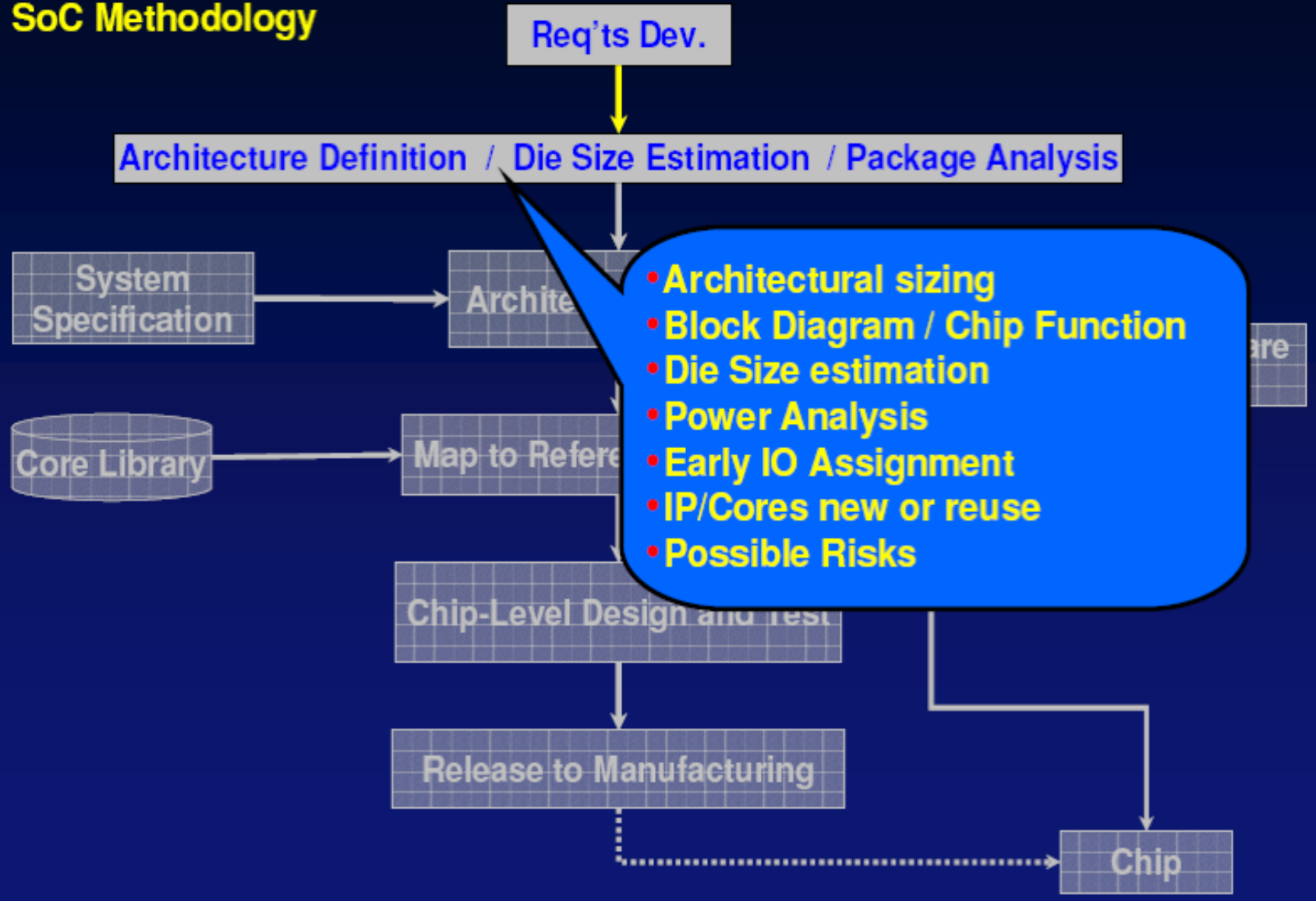
SOC Methodology Evolving ...

SoC Methodology



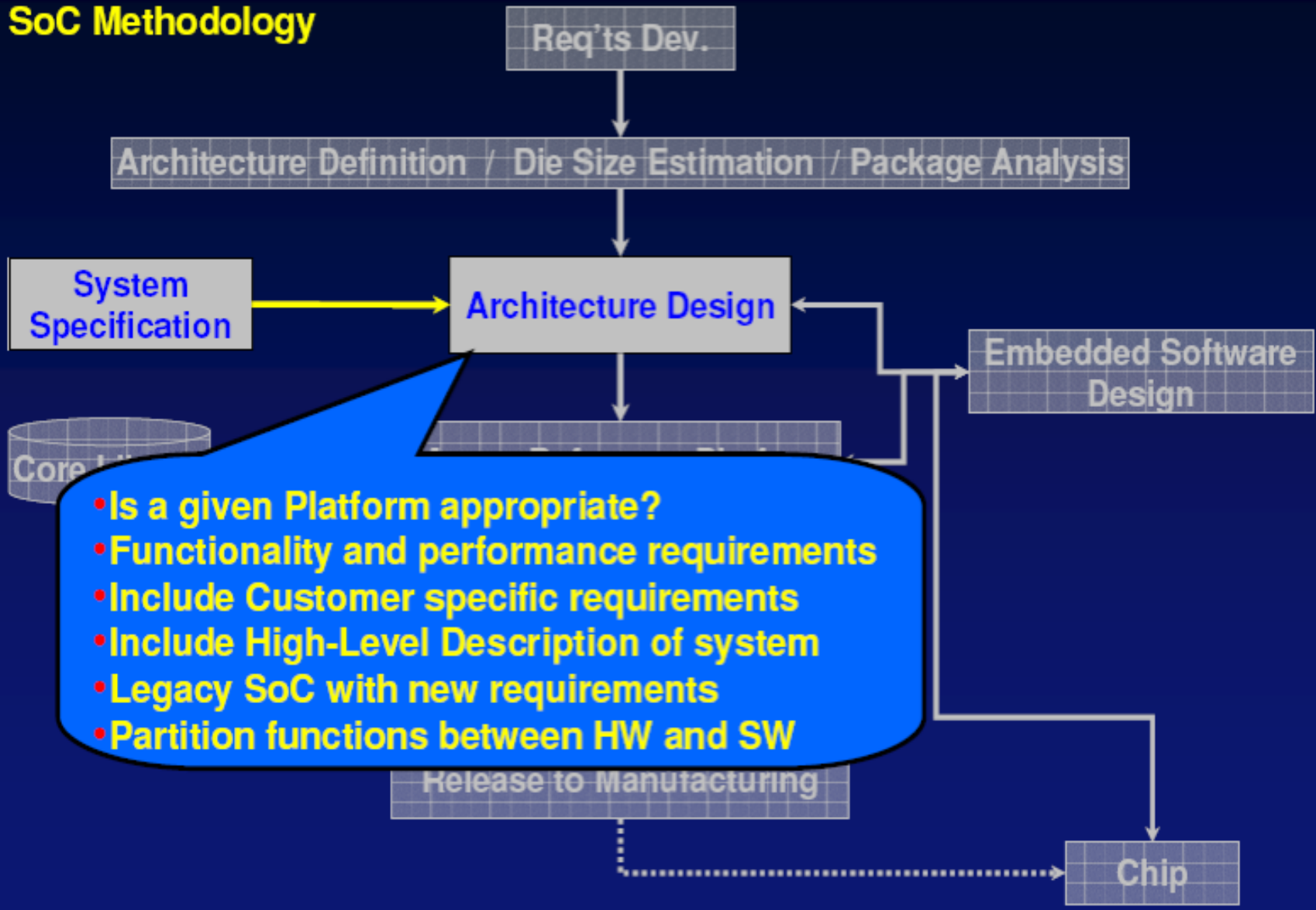
How to Design an SOC

SoC Methodology



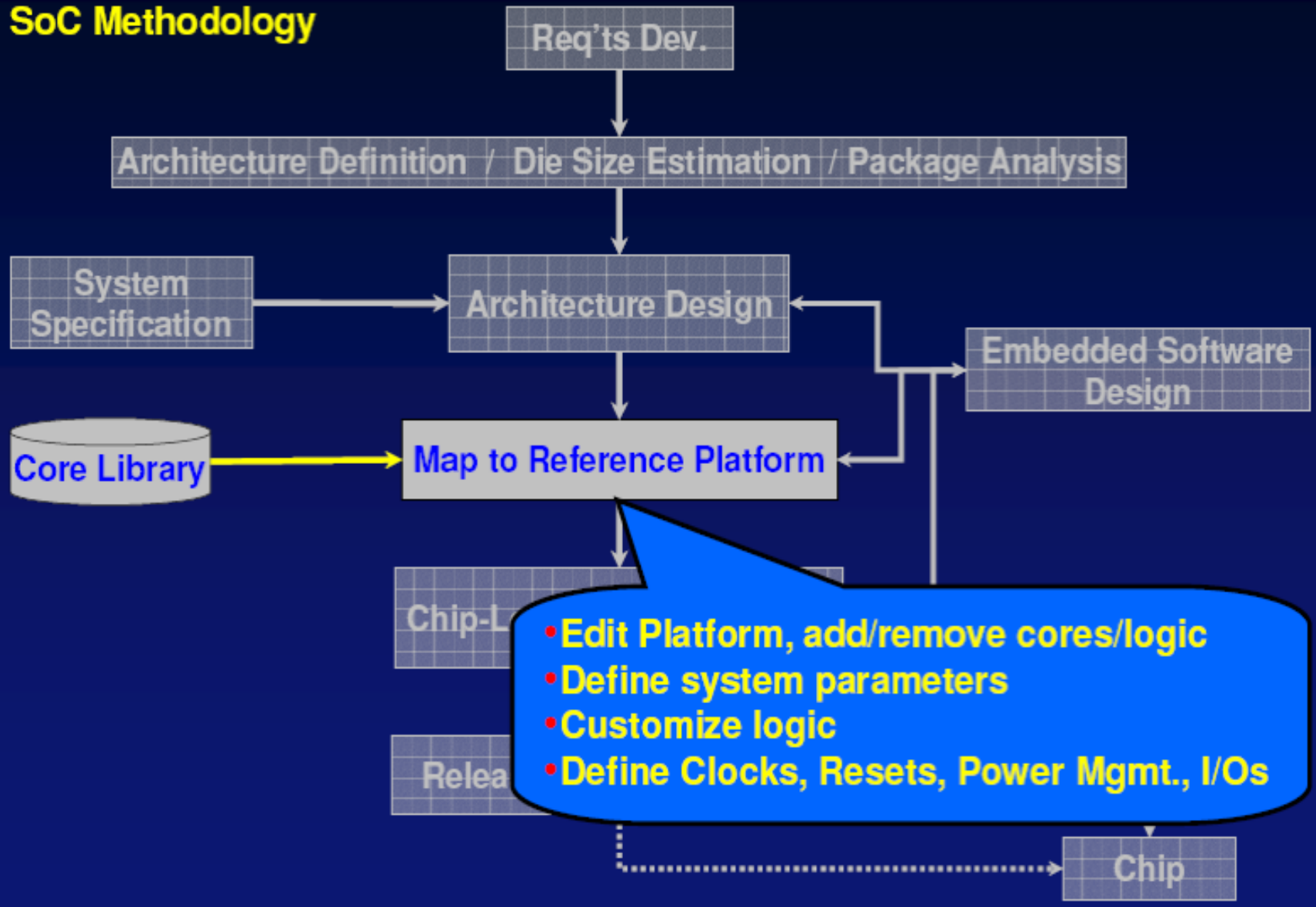
How to Design an SOC

SoC Methodology



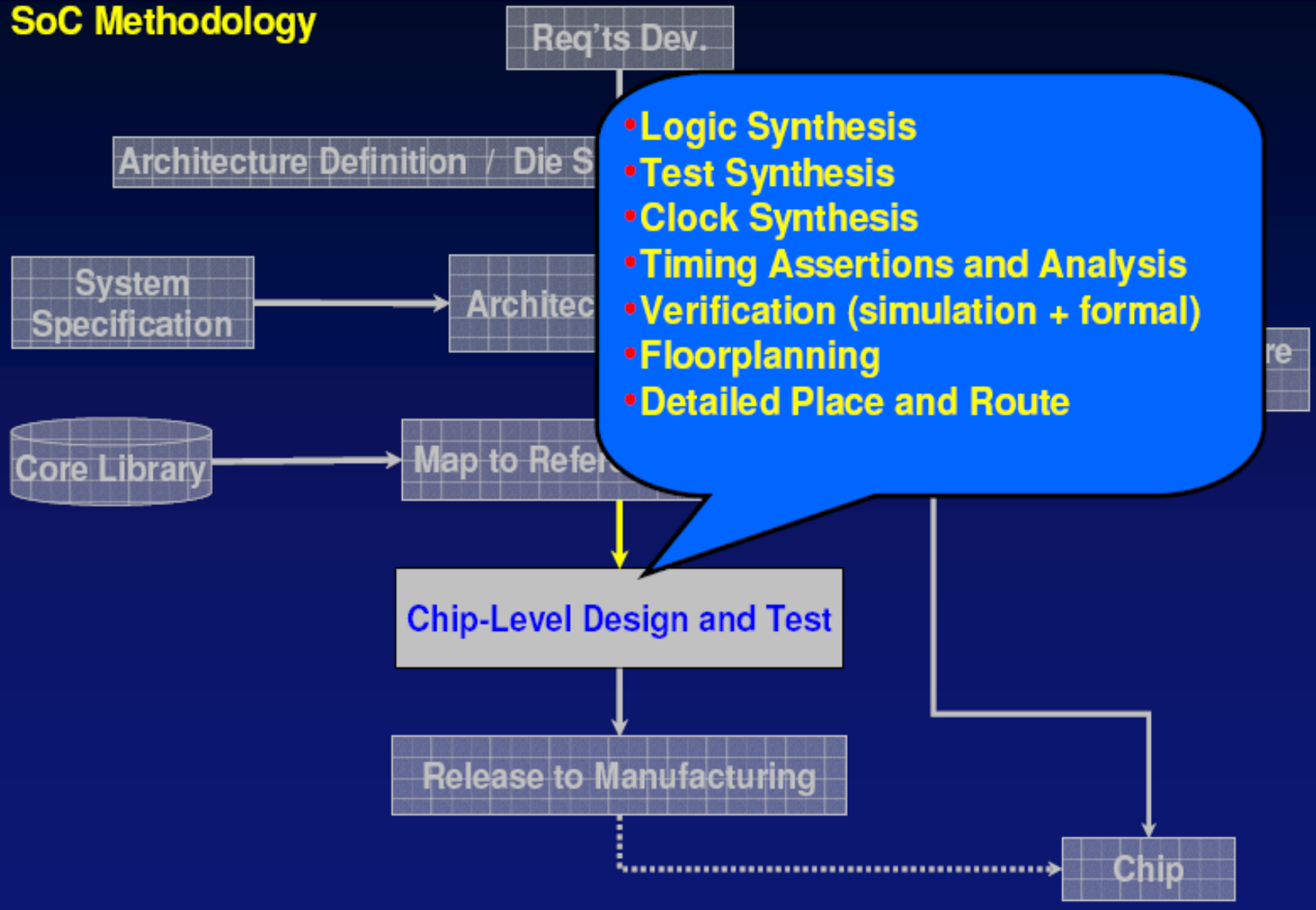
How to Design an SOC

SoC Methodology



How to Design an SOC

SoC Methodology



Key to SOC Design Process

- **Iteration** is an inevitable part of the design process
- The problem is **how large the loop is**
- Goal
 - **Minimize the overall design time**
- But How
 - Planned for iterations
 - Minimize iteration numbers
 - especially major loops (Spec to chip)
 - Local loop is preferred
 - coding, verifying, synthesizing small blocks
 - IP clearly help due to pre-verified
 - Parameterized blocks offer more tradeoff between area, performance and functionality
- Carefully designed **spec** is the best way to minimize the loops

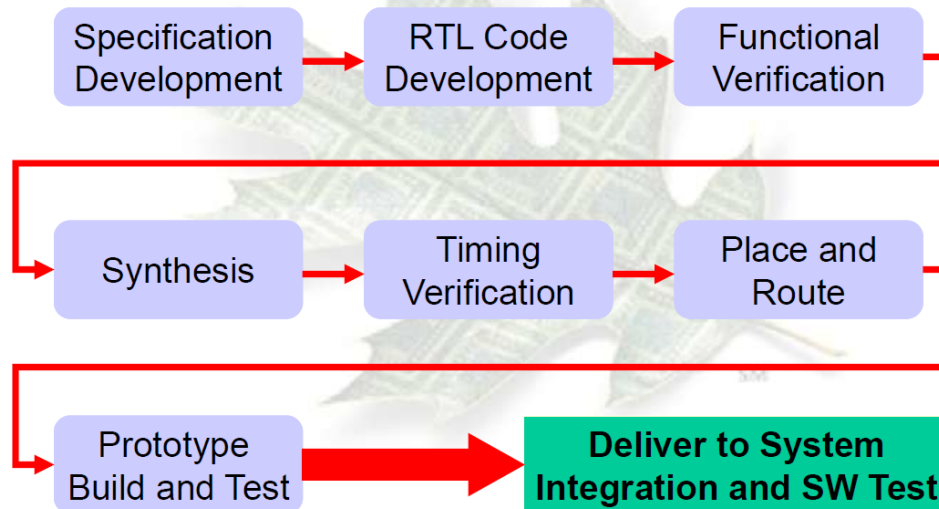
System Design Flow

- SoC challenges force chip designers to alter design flow
 - *Waterfall* model to *spiral* model
 - *Top-down* methodology to *Top-down/Bottom-up* combination

Waterfall Model

- **Step-to-step handoff**
- **Fewer feedback paths in the flow**
 - Possibilities for re-iteration exist

Waterfall Methodology



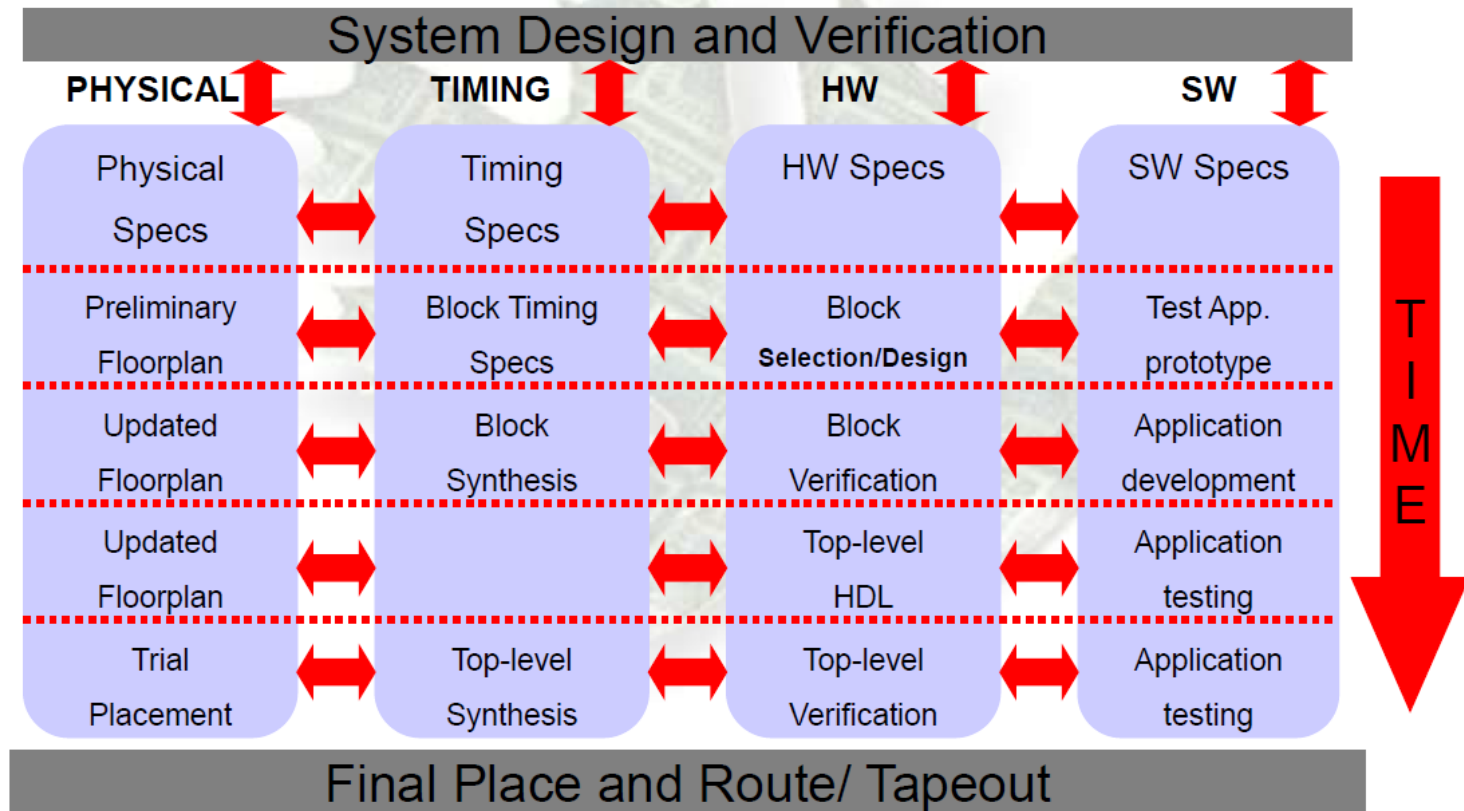
Waterfall Flow Limitations

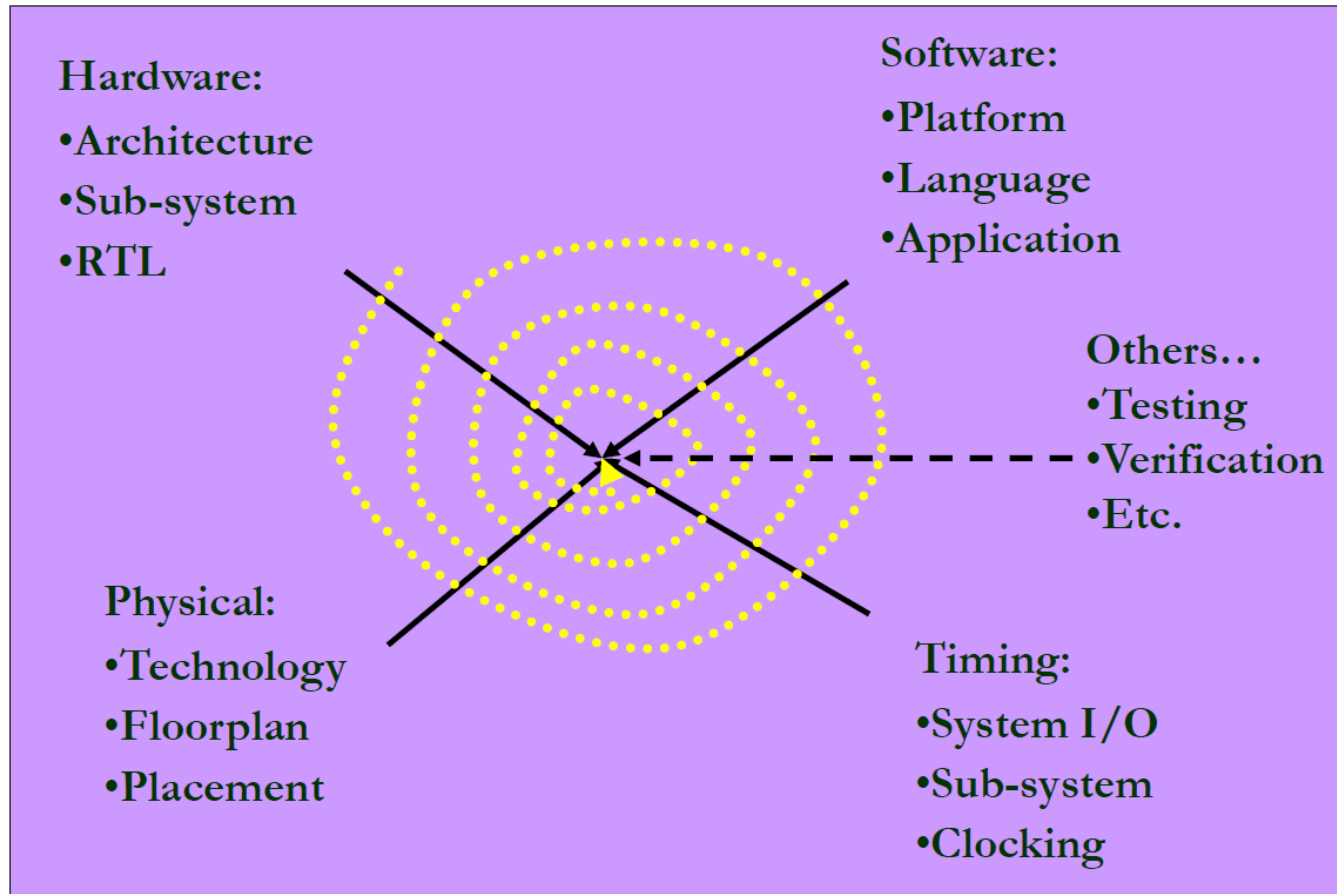
- **Handoff between teams rarely clean**
 - May have to tell the system designers algorithm will not work
- **Doesn't work for large, deep submicrometer designs**
 - Software and hardware must be developed concurrently
 - Physical design issues must be considered early to meet performance goals

Spiral Development Model

- **Response to increased complexity and shorter time-to-market pressure**
- **Multiple aspects of design are worked on simultaneously**
- **Each area is incrementally improved together as the entire project is completed**

Spiral Methodology





Spiral Design Characteristics

- **Concurrent HW & SW development**
- **Parallel verification and synthesis**
- **Floorplanning and place-and-route included in the synthesis process**
- **Modules developed only if a predesigned hard or soft macro is not available**
- **Planned iteration throughout**